

Experiment No.	4
Experiment Title	Binary Classification using Linear and Kernel-Based Models: Logistic Regression and Support Vector Machines on the Spambase Dataset
Student Name	S.S.Janane
Register Number	3122247001024
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	02/08/26

1 Objective

The aim of this experiment is to build a spam-detection system to classify emails as spam or ham using Logistic Regression and Support Vector Machine (SVM) classifiers and to analyze the effect of hyperparameter tuning on classification performance

2 Problem Statement

Spam filtering is a classic binary classification problem: given a set of numeric features extracted from an email's content , how often certain words or characters appear, and how long runs of capital letters tend to be , the task is to predict whether the email is spam (label 1) or legitimate, i.e ham (label 0). The input both models is a 57-dimensional feature vector per email, and the output is a single binary label. The objective is to train Logistic Regression and SVM classifiers on data, tune their hyperparameters using both Grid Search and Randomized Search, and determine which model generalizes better while also understanding *why* it does.

3 Dataset Description

Dataset Name	Spambase
Dataset Source	UCI Machine Learning Repository (also mirrored on Kaggle); the same CSV file used in an earlier experiment on this dataset was reused here
Number of Samples	4601 emails
Number of Features	57 numeric predictors , word-frequency features, character-frequency features (e.g. the frequency of the dollar sign or exclamation mark), and three features describing runs of consecutive capital letters
Number of Classes	2 , ham (0) and spam (1)
Missing Values	None found. 391 duplicate rows were present but intentionally retained (see Section 5)

Train-Test Split	80% / 20%, stratified on the class label (3680 training emails, 921 test emails), using a fixed random seed for reproducibility
------------------	---

4 Exploratory Data Analysis

Before touching any models, it helps to look at the data and see whether the patterns one would intuitively expect from a spam filter, lots of exclamation marks, dollar signs, and shouty capital letters, actually show up numerically. The dataset itself is very clean: calling an information summary on it confirmed all 58 columns (57 features plus the class label) are populated and numeric, so there was no type conversion or parsing to worry about.

Missing Value Analysis

As expected, the dataset has no missing values. Since Spambase consists of pre-extracted feature frequencies instead of raw text, every record already has a complete set of computed values. The only data quality issue identified was the presence of 391 duplicate rows. At first glance this looks like something that should be cleaned up, but it isn't really a data quality issue here: because every feature is a frequency (the percentage of words in an email matching a keyword), it is entirely plausible for two short, similarly templated emails to land on the exact same feature vector without actually being the same email. Removing these rows would have thrown away legitimate data points for no good reason, so they were deliberately left in.

Class Distribution

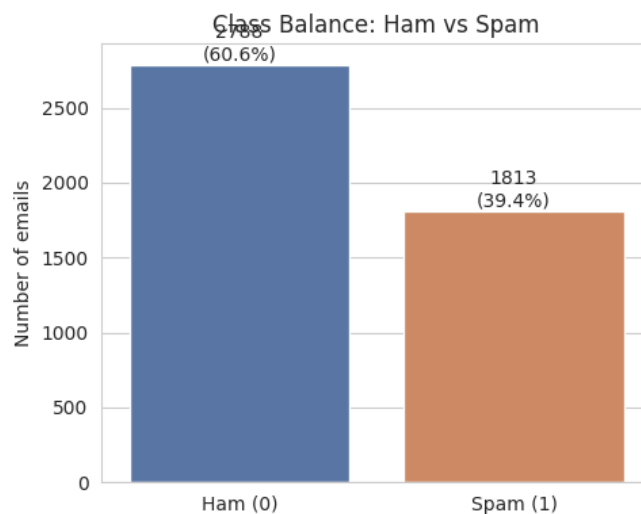


Figure 1: Class balance between ham and spam emails in the Spambase dataset.

Inference:

Totally 4601 emails, 2788 (about 60.6%) are ham and 1813 (about 39.4%) are spam. This is a mild imbalance rather than a severe one, so accuracy remains a reasonably trustworthy metric here and there was no real need to resample or reweight the classes, stratified splitting was used everywhere in this experiment , for the train-test split, for cross-validation, and inside the hyperparameter search , purely to make sure this roughly 60/40 ratio is preserved consistently rather than fluctuating across folds by chance.

Statistical Summary

Feature	Mean	Std. Dev.	Min	Median	Max
word_freq_free	0.249	0.826	0.000	0.000	20.000
word_freq_your	0.810	1.201	0.000	0.220	11.110
char_freq_\$	0.076	0.246	0.000	0.000	6.003
capital_run_length_average	5.192	31.729	1.000	2.276	1102.500
capital_run_length_total	283.289	606.348	1.000	95.000	15841.000

Table 2: Summary statistics for a few representative features.

Inference:

Even this small slice of the summary statistics tells a story. The heavy skew is obvious , for a feature like word_freq_free, the median is 0 but the mean is 0.249 and the maximum reaches 20, meaning most emails never use the word “free” at all, but a small subset use it extremely often. The capital-letter-run features show the same pattern on a much larger scale, with capital_run_length_total ranging all the way up to nearly 16,000 in the most extreme email. This kind of spread across wildly different scales is exactly why feature standardization was necessary before fitting either Logistic Regression or SVM, both of which are sensitive to the relative magnitude of the input features.

Correlation with the Target

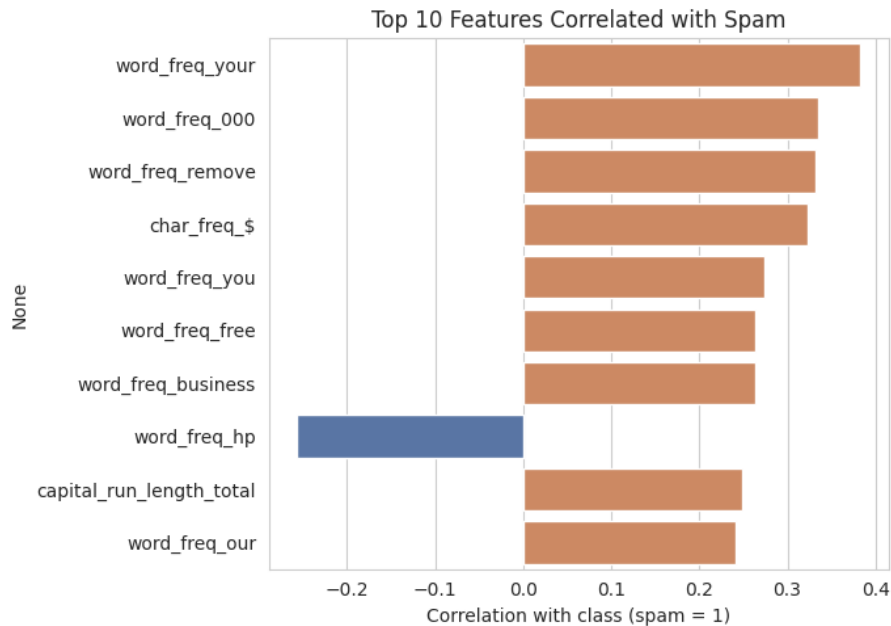


Figure 2: Top 10 features most strongly correlated with the spam label.

Inference:

The features most strongly associated with spam were word freq your, word freq 000, word freq remove, and char freq, each showing a positive correlation greater than 0.32. On the other hand word freq hp had the strongest negative correlation with the spam label. This is not surprising, as the Spambase dataset was collected from a corporate Hewlett-Packard mailbox, meaning legitimate internal emails were much more likely to contain the term "hp" than spam messages. Although none of these individual correlations are particularly strong—the highest is only about 0.38—they suggest that many moderately informative features, when considered together, can effectively distinguish spam from legitimate emails. This expectation was reflected in the performance of the trained models.

Feature Distribution by Class

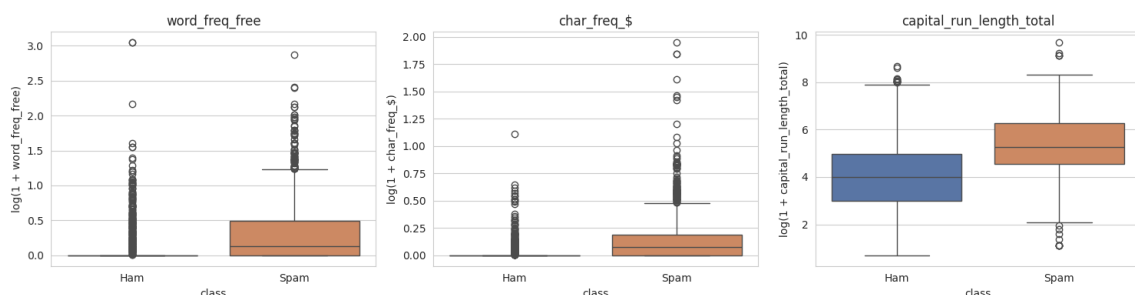


Figure 3: Distribution of three key features (log-scaled) split by class.

Inference:

Plotting these three features on a log scale (to tame their heavy skew) makes the class separation visually obvious. Spam emails clearly use the word “free” and the dollar sign far more often than ham emails, and their capital-letter runs are noticeably longer on average, the mean `capital_run_length_total` sits around 471 for spam versus only about 161 for ham. None of this is a surprising result for anyone who has ever glanced at a spam folder, but it was still a useful sanity check to confirm that the data actually shows real-world spam characteristics before any modeling was attempted. Seeing the boxes for spam sit visibly higher than ham across all three features gave good confidence that a classifier would have genuine signal to learn from, not just noise.

5 Data Preprocessing

The preprocessing pipeline for this experiment was fairly lightweight, mostly because Spambase arrives as a clean, pre-extracted numeric table rather than raw text.

- **Missing value handling:** Not required, the dataset contained zero missing values.
- **Duplicate rows:** 391 duplicate rows were identified but intentionally kept, they represent genuinely repeated feature vectors rather than corrupted entries (see Section 4).
- **Encoding:** Not required, all 57 predictors are already numeric (frequencies and counts), and the target is already a binary integer label.
- **Feature scaling:** A standard scaler was fit on the training features only (subtracting the mean and dividing by the standard deviation for each column) and then applied to both the training and test sets. Fitting the scaler exclusively on the training data was a deliberate choice to avoid leaking any information from the test set into the preprocessing step.
- **Train-test split:** An 80/20 split was used, stratified on the class label so that both the training and test sets preserve the roughly 60/40 ham-to-spam ratio seen in the full dataset. To make the results reproducible, a random seed of 42 was used consistently for the data split, model initialization, and cross-validation process.
- **Feature engineering:** No additional engineered features were introduced, the 57 original Spambase features were used as-is, since they are already fairly informative hand-crafted frequency statistics.

6 Machine Learning Algorithm

Logistic Regression

Working principle. Logistic Regression models the probability that a given email is spam as a function of a weighted sum of its features, squashed through a sigmoid function

so the output always lies between 0 and 1. If that probability crosses a threshold (0.5 by default), the email is classified as spam. Because the decision boundary is ultimately a linear combination of the inputs, the model is essentially drawing a straight hyperplane through the 57-dimensional feature space to separate the two classes.

Mathematical formulation: For an input vector x with weights w and bias b , the model computes

$$p(y = 1 | x) = \sigma(w^\top x + b) = \frac{1}{1 + e^{-(w^\top x + b)}}$$

and the weights are learned by minimizing the regularized log-loss over the training set:

$$\mathcal{L}(w, b) = -\frac{1}{n} \sum_{i=1}^n \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right] + \lambda R(w)$$

where $R(w)$ is either the L1 penalty ($\|w\|_1$, which encourages sparse coefficients) or the L2 penalty ($\|w\|_2^2$, which shrinks all coefficients smoothly), and $\lambda = 1/C$ controls how strongly the model is regularized.

Advantages: It is fast to train, its coefficients are directly interpretable as the influence of each feature on the log-odds of spam, and it tends to work very well when the classes are close to linearly separable, which turned out to be roughly true for this dataset.

Limitations: It cannot natively capture non-linear decision boundaries, it can be sensitive to strongly correlated (multicollinear) features, and its performance depends on properly scaled inputs, all of which is why the scaling step earlier in the pipeline mattered.

Support Vector Machine

Working principle. An SVM tries to find the hyperplane that separates the two classes with the widest possible margin, relying only on the closest points to the boundary, the “support vectors”, to define it. When the classes are not linearly separable in the original feature space, a kernel function implicitly projects the data into a higher-dimensional space where a linear separator becomes possible, without ever having to compute that projection explicitly (the so-called “kernel trick”).

Mathematical formulation : The soft-margin SVM optimization problem is

$$\min_{w, b, \xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i \quad \text{subject to} \quad y_i(w^\top \phi(x_i) + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

where $\phi(\cdot)$ is the (possibly implicit) feature mapping induced by the kernel, ξ_i are slack variables allowing some misclassification, and C controls the trade-off between a wide margin and fewer training errors. In this experiment, four kernels were compared: the linear kernel $K(x_i, x_j) = x_i^\top x_j$, the polynomial kernel $K(x_i, x_j) = (\gamma x_i^\top x_j + r)^d$, the radial basis function (RBF) kernel $K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$, and the sigmoid kernel $K(x_i, x_j) = \tanh(\gamma x_i^\top x_j + r)$.

Advantages : SVMs handle high-dimensional feature spaces well, the kernel trick gives them the flexibility to model non-linear boundaries when needed, and with a properly chosen C they tend to generalize well without overfitting.

Limitations: They are noticeably less interpretable than a linear model , there is no simple way to say “this feature increases the spam probability by this much” once a non-linear kernel is involved , and they can be more computationally expensive to tune, since the hyperparameter search space (kernel, C , γ , and degree for polynomial kernels) is considerably larger than for Logistic Regression, which was reflected directly in the search times recorded in this experiment.

7 Experimental Setup

Python Version	3.12
Scikit-Learn Version	1.6.1
IDE	Jupyter Notebook
Operating System	Linux
Hardware	Colab standard CPU runtime

8 Hyperparameter Selection

Two separate search strategies , Grid Search and Randomized Search, both using 5-fold cross-validation on the training set , were used to tune Logistic Regression, while SVM was tuned with Grid Search alone across all four kernels simultaneously.

Model	Parameter	Search Space
Logistic Regression	Penalty	L1, L2
	C	0.01, 0.1, 1, 10, 100 (Grid); log-uniform(0.01, 100) (Randomized)
	Solver	liblinear, saga
SVM	Kernel	linear, RBF, sigmoid, polynomial
	C	0.1, 1, 10, 100
	Gamma	scale, auto
	Degree (polynomial only)	2, 3, 4

Table 3: Hyperparameter search spaces used for each model.

Model	Search Method	Best Parameters	Best CV Accuracy
Logistic Regression	Grid Search	C=10, penalty=L2, solver=saga	0.9239
Logistic Regression	Randomized Search	C $\approx$ 2.34, penalty=L1, solver=liblinear	0.9242
SVM	Grid Search	kernel=RBF, C=10, gamma=scale	0.9332

Table 4: Best hyperparameters found for each model and search method.

The two Logistic Regression searches landed on almost identical cross-validation accuracy , 0.9239 for Grid Search versus 0.9242 for Randomized Search , despite picking visibly different-looking winners: Grid Search preferred L2 regularization with a fairly permissive C of 10, while Randomized Search preferred L1 with C around 2.34, also on the light side. The Randomized Search result was carried forward purely because it scored a fraction higher, but realistically the two are close enough to call it a tie. The more interesting takeaway is that both searches converged toward light regularization rather than a heavily constrained model, suggesting the 57 standardized features are not especially noisy or redundant. In terms of search cost, Grid Search took about 24.6 seconds and Randomized Search about 12.1 seconds for Logistic Regression, while the SVM grid search , covering four kernels and their respective parameter grids , took considerably longer at around 106 seconds, which is expected given the much larger combined search space.

9 Results

Metric	Logistic Regression (tuned)	SVM , RBF (tuned)
Accuracy	0.9262	0.9207
Precision	0.9178	0.9143
Recall	0.8926	0.8815
F1-score	0.9050	0.8976
ROC-AUC	Not computed in this experiment	Not computed in this experiment

Table 5: Held-out test set performance for both tuned classifiers.

For additional context, the four SVM kernels were also compared before any tuning, using scikit-learn’s default settings for each:

Kernel	Accuracy	F1-score	Training Time (s)
Linear	0.9294	0.9093	0.47
Polynomial (degree 3)	0.7796	0.6220	0.41
RBF	0.9273	0.9055	0.20
Sigmoid	0.8849	0.8528	0.25

Table 6: Untuned SVM performance across all four kernels on the test set.

Somewhat counter-intuitively, on this single held-out test split, tuned Logistic Regression edges out tuned SVM on every metric. This is worth keeping in mind while reading the visualizations and cross-validation results in the next two sections, since the picture changes once averaged over multiple folds.

10 Result Visualization

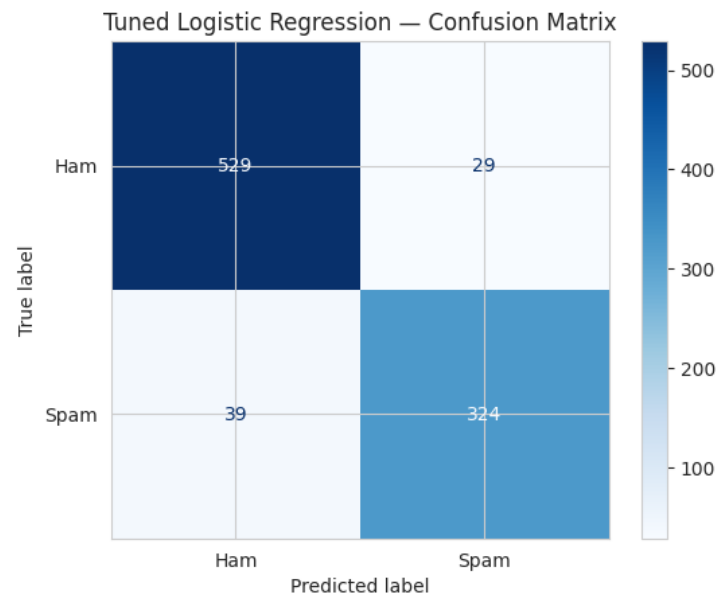


Figure 4: Confusion matrix for the tuned Logistic Regression classifier on the test set.

Inference:

From 921 test emails, the tuned Logistic Regression model correctly identifies the large majority of both classes, with a slightly stronger recall on ham (about 95%) than on spam (about 89%). The precision on spam predictions is high at roughly 92%, implying that when the model does flag an email as spam, it is usually right, an important property for a spam filter, since false positives (legitimate mail marked as spam) tend to be more costly to a user than the occasional missed spam email. The confusion matrix confirms this asymmetry is mild rather than severe, which is consistent with the overall F1-score of 0.905.

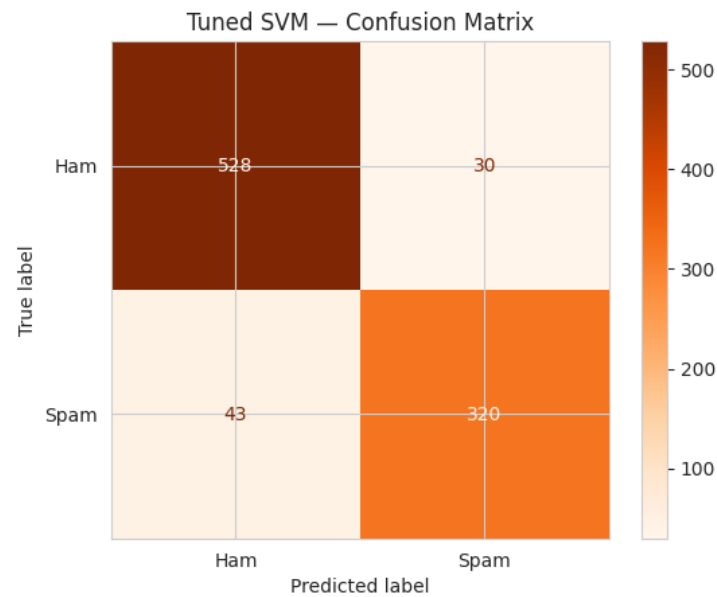


Figure 5: Confusion matrix for the tuned SVM (RBF kernel) classifier on the test set.

Inference:

The SVM's confusion matrix looks very similar in shape to Logistic Regression's, again siding towards ham recall (about 95%) over spam recall (about 88%). The two models misclassify a broadly similar number of emails, with SVM making slightly more errors overall on this particular split , 73 misclassifications versus Logistic Regression's roughly 68. This closeness is exactly why a single train-test split is not enough to declare a clear winner and is the motivation for the cross-validation comparison that follows.

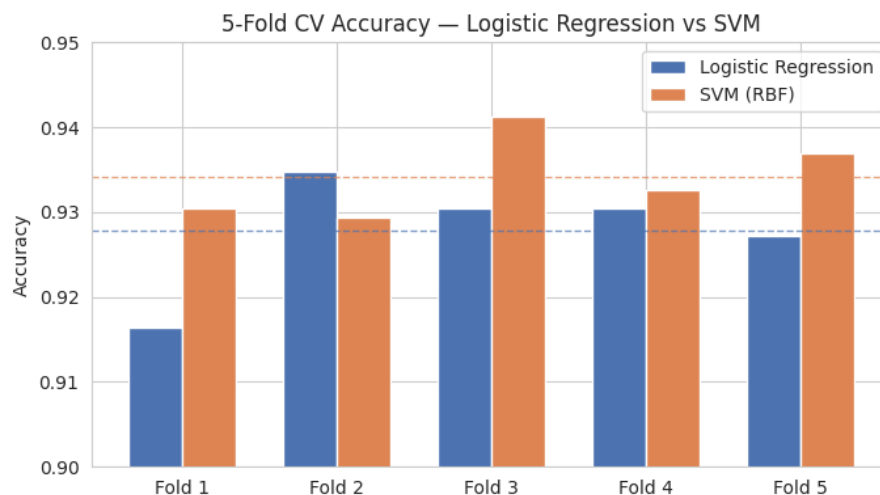


Figure 6: 5-fold stratified cross-validation accuracy for both tuned models.

Inference:

Averaged over five folds, SVM comes out ahead of Logistic Regression , roughly 93.4% versus 92.8% , and it is also the more consistent of the two, with a tighter spread of accuracy across folds. This is a notable reversal from the single test-split result in the

previous section, where Logistic Regression actually scored higher. It is a good practical reminder that any one train-test split can be a little noisy, especially with a dataset of this size, and that cross-validated averages are generally the more trustworthy number to report when two models are this close together.

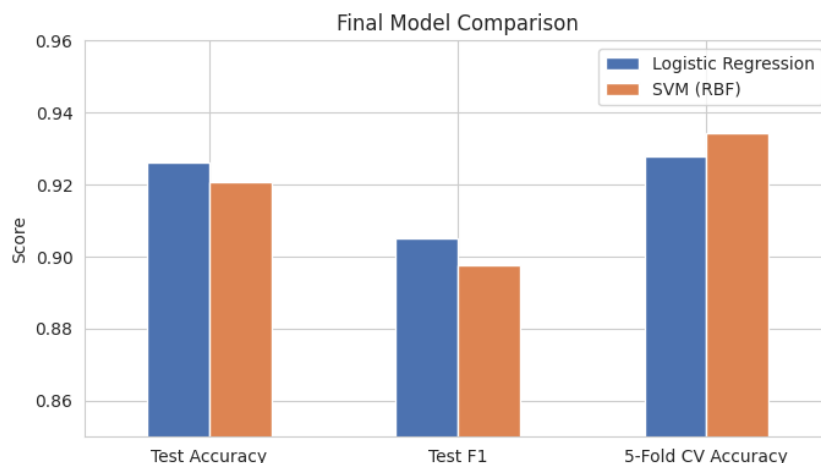


Figure 7: Final comparison of test accuracy, test F1-score, and 5-fold CV accuracy for both models.

Inference:

This chart lines up all three headline numbers side by side and makes the overall trade-off easy to see at a glance: Logistic Regression is slightly ahead on the single test split (both accuracy and F1), while SVM pulls ahead on the cross-validated average, which is the more reliable of the two figures. The gap in either direction is small, on the order of half a percentage point to just under one percentage point, so in practical terms, both models performance are at a very similar level on this dataset, and the choice between them may come down as much to interpretability needs as to raw accuracy.

11 Discussion

Observations stood out while working through this experiment.

On which model actually wins: It depends on which number you trust. The single held-out test split sides towards Logistic Regression (0.9262 vs 0.9207 accuracy), but the 5-fold cross-validation average favouring SVM (0.9341 vs 0.9278), and is also the more stable estimate of the two. Taking the CV result as the fairer comparison, SVM with the RBF kernel comes out as the marginally better classifier overall, though the margin is small enough that Logistic Regression remains a very strong, and can be interpreted.

On regularization: Both Logistic Regression searches converged toward fairly light regularization ($C=10$ for Grid Search, $C=2.34$ (approx.) for Randomized Search) rather than the heavily constrained end of the search space. This suggests the 57 standardized features are not particularly noisy or redundant, so the model benefits from being given more freedom rather than being shrunk aggressively. L1's sparsity effect is present in the winning Randomized Search configuration, but it does not translate into a meaningfully

higher accuracy than L2 once C is properly tuned for each, the two regularization styles end up performing almost identically here.

On kernel behaviour: Linear and RBF kernels were both immediately strong even with default settings, which is a good sign that spam versus ham is close to linearly separable once the features are standardized. Sigmoid trailed back noticeably, and the polynomial kernel was the real surprise, it collapsed to about 78% accuracy with its default degree-3 setting, with recall taking the biggest hit. That said, this looks like a default-hyperparameter problem rather than the kernel itself being unsuitable for the task; even after tuning, though, the grid search still preferred RBF over any tuned polynomial configuration, suggesting a smooth non-linear boundary simply fits this data better than a rigid polynomial one.

On bias and variance: Logistic Regression, being the simpler linear model, sits on the higher-bias, lower-variance side of the spectrum, fast and stable, but unable to bend its decision boundary. SVM with an RBF kernel can capture more non-linear structure (lower bias) at the cost of being more sensitive to its C and gamma settings (higher variance if left untuned). The fact that the untuned RBF SVM was already close to its fully tuned performance suggests this particular dataset doesn't demand very aggressive tuning, whereas the polynomial kernel's collapse under default settings is a textbook example of what high-variance, under-tuned behaviour looks like.

12 Conclusion

This experiment set out to compare a linear probabilistic classifier against a margin-based, kernel-capable one on the task of spam detection, and to see how much hyperparameter tuning actually matters along the way. Both Logistic Regression and SVM (with an RBF kernel) ended up performing at a very similar level, in the low-to-mid 90s percentage-wise, with neither model dramatically outperforming the other. Cross-validation, being the more reliable measure over a single split, gave a slight edge to SVM, but Logistic Regression's interpretability and near-identical accuracy make it a perfectly reasonable choice in a setting where explaining predictions matters. Perhaps the most useful lesson from this experiment wasn't about spam filtering specifically, but about evaluation methodology in general: a single train-test split disagreed with the cross-validated average on which model was better, which is a strong argument for always checking results across multiple folds before declaring a winner.

Learning Outcomes

- Gained a clearer, hands-on sense of how probabilistic (Logistic Regression) and margin-based (SVM) classifiers actually differ in practice.
- Applied both Grid Search and Randomized Search for hyperparameter tuning, and saw how close their results can land on a search space of this size, close enough that the "winner" came down to a 0.0003 difference in cross-validation accuracy.
- Practiced evaluating classifiers properly rather than relying on accuracy alone, precision, recall, F1-score, and the confusion matrices told a much more complete

story, especially for the polynomial kernel's recall collapse.

- Learned to trust cross-validated averages over a single train-test split when two models are close, since the single split and the CV average actually disagreed here on which model came out ahead.

13 References

- [1] scikit-learn developers, “Logistic Regression,” scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
- [2] scikit-learn developers, “Support Vector Machines,” scikit-learn documentation. [Online]. Available: <https://scikit-learn.org/stable/modules/svm.html>
- [3] scikit-learn developers, “Tuning the hyper-parameters of an estimator,” scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/modules/grid_search.html
- [4] Kaggle, “Spambase Dataset.” [Online]. Available: <https://www.kaggle.com/datasets/colormap/spambase>
- [5] M. Hopkins, E. Reeber, G. Forman, and J. Suermondt, “Spambase Data Set,” UCI Machine Learning Repository. [Online]. Available: <https://archive.ics.uci.edu/dataset/94/spambase>